YOUR TAKE-HOME CHALLENGE

Build a cold-email engine that never loses track of a contact.

Thank you for clearing the first round. This assignment is close to the work you would do at MnemioAI: take a messy business process, automate it with AI, and make it reliable at scale. **We care more about a system that works than about how much you build.**

TIME TO SUBMIT **5 days from receipt**	EXPECTED EFFORT **About 8 to 10 hours**	LANGUAGE AND TOOLS **Your choice**
AI **OpenRouter free models**	EMAIL SENDING **Mocked, never real**	SUBMIT **Public GitHub repo**

The scenario

A sales team is running cold outreach to **10,000 prospects**. Each prospect gets a first email and up to three follow-ups. At any moment the team needs to know who has been emailed, who replied, who bounced, what is due next, and who should no longer be contacted. Today this lives in spreadsheets and people's memory, and contacts fall through the cracks or get emailed twice.

Your job is to build the workflow that sends this sequence and tracks every contact through it. You may use any existing tool (for example n8n, an open-source CRM, a database and a job queue) or build your own with the help of AI. Tell us why you chose your approach.



Replied
Sequence stops at once. Reply is recorded and classified.

Bounced
Sequence stops. Contact marked bounced, never retried as valid.

Unsubscribed
Added to a suppression list. Never contacted again.

The happy path runs left to right. A reply, bounce or unsubscribe at any step ends the sequence. If the last follow-up gets no reply within its wait period, the contact is deactivated.

Ground rules. Never send real emails. Use OpenRouter's free models; you should not need any paid API. Do not commit secrets. AI coding assistants are welcome, but the work must be your own and you should be ready to walk us through every part of it in the follow-up interview. Questions? Email **vikrant@mnemioai.com**.

1 Seed data: about 10,000 contacts, with history

A script that creates around 10,000 contacts (name, email, company, role, timezone) and gives each one a **random status** in the sequence, along with the matching history. A contact on follow-up 2 must have records for the first email and follow-up 1, with sensible dates. Use a fixed random seed so we get the same data when we run it. Your workflow then runs on top of this data, so we can see that it works at 10,000 contacts and more. Example statuses:

STATUS	MEANING
`new`	Not contacted yet
`first_sent`	First email sent, waiting for a reply
`follow_up_1_sent … follow_up_3_sent`	Follow-up sent, waiting for a reply
`replied`	Prospect replied; sequence stopped
`bounced`	Email bounced; sequence stopped
`unsubscribed`	Asked not to be contacted; suppressed
`deactivated`	All follow-ups sent with no reply

2 The sequence workflow

A first email plus three follow-ups, with configurable waits between them (for example 3, 5 and 7 days). Each run finds the contacts that are due, writes the email, sends it through your mock sender, records what happened and schedules the next step. **Running the workflow twice must never send the same step to the same contact twice.**

3 Mock sending, mock replies, and a clock you control

Do not send real emails. Build a mock email provider that accepts or bounces messages at configurable rates, and simulates replies arriving: interested, not interested, out of office, and unsubscribe requests. Give us a way to move time forward, so we can watch weeks of a campaign play out in minutes.

4 Tracking and lifecycle rules

- Track for every contact: emails sent, bounces, replies, follow-ups sent, and the next action and its date.
- A reply stops the sequence. Bounces and unsubscribes stop it and are never contacted again.
- When all follow-ups are sent and the last wait passes without a reply, mark the contact **deactivated**.

5 AI with OpenRouter free models

Use free models from openrouter.ai (model IDs ending in `:free`) to write and personalise the emails and to classify replies. Free models are rate-limited, and 10,000 contacts will hit those limits: show us how you design around it (for example templates per segment, caching, batching, retries and a fallback). Keep your key in a `.env` file and commit only a `.env.example`.

6 Reporting, at full scale

A command, endpoint or page that shows counts by status, sent today, due today and tomorrow, reply and bounce rates, and deactivated contacts. Run the whole thing on the full 10,000-contact seed and include in your README how long it took on your machine and the final counts.

Bonus points

Warm-up

- Several mock sending mailboxes, each with a daily limit that ramps up over time
- Sends spread across mailboxes and across the working day, not in one burst
- A clear view of each mailbox's warm-up schedule and today's usage

Spam management

- Pause a mailbox automatically when its bounce or complaint rate crosses a threshold
- Suppression list, unsubscribe link and a List-Unsubscribe header
- Content checks before sending; sending windows by the prospect's timezone; limits per recipient domain

Also welcome: a simple dashboard to view campaigns and contact states, automated tests, and a one-command setup (for example Docker Compose).

What to submit

A **public GitHub repository**, with a link emailed to **vikrant@mnemioai.com** within 5 days, subject `Take-home: your name`. We review the last commit made before your deadline. Your README will be read by a person, so write it for one. It should cover:

- What the system does, in one short paragraph
- How to set it up and run it in a few commands: seed, run the workflow, move time forward, see the report
- Architecture and data model, and why you chose your tools
- The lifecycle rules, and how you stop double sends
- Where AI is used, which models, and how you handle rate limits
- Results of your 10,000-contact run: time taken and final counts
- Trade-offs, known limitations, and what you would build next

How we evaluate

- **It works.** We clone your repository, follow your README on a clean machine, seed 10,000 contacts and run the workflow. The lifecycle is correct and re-runs do not double send.
- **Tracking is complete.** Sent, replied, bounced, unsubscribed, follow-ups and deactivation are all accurate and easy to inspect.
- **The README is clear.** A person can understand and run your system without asking you anything.
- **Engineering judgement.** How you handle scale, rate limits, failures and your data model.
- **Bonus.** Warm-up and spam management, a dashboard, and tests.